



HeeraMon

Development Roadmap

Development Roadmap Document

Date of Preparation: September 30, 2026

Document ID	HM-DOC-15
Title	Development Roadmap
Classification	Development Roadmap Document
Status	Draft
Version	1.0
Owner	Product / Engineering
Audience	All project contributors
Related Documents	Product Vision (HM-DOC-01), Technical Architecture (HM-DOC-07)

Contents

1 Purpose	2
2 Development Strategy	2
3 Milestone 0 — Project Foundation	2
4 Milestone 1 — Conversational Core	3
5 Milestone 2 — Job Ingestion	3
6 Milestone 3 — Matching and Job Presentation	4
7 Milestone 4 — Continuous Search	4
8 Milestone 5 — Preference Learning	5
9 Milestone 6 — Application Assistance	5
10 Milestone 7 — Hardening and Release	5
11 Recommended Build Order	6
12 Definition of Completion	6
13 What Should Not Delay Release	7
14 Development Rule	7
15 Document Control	8

1 Purpose

This document defines the implementation sequence for HeeraMon.

The roadmap is designed to produce a working end-to-end product as early as possible while keeping the architecture suitable for future expansion.

The goal is not to build every planned capability at once. Each milestone should produce a usable and testable increment.

2 Development Strategy

HeeraMon should be developed around the core loop:

**Describe → Understand → Search → Evaluate → Inform → Discuss →
Decide → Apply**

Development should prioritize the smallest complete version of this loop before adding advanced features.

The following principles apply:

- Build vertically before building broadly.
- Prefer working functionality over placeholder infrastructure.
- Keep deterministic logic outside the LLM where practical.
- Keep external integrations behind adapters.
- Test critical behavior as it is implemented.
- Avoid building later-stage infrastructure before the core product demonstrates value.
- Keep the system deployable throughout development.

3 Milestone 0 — Project Foundation

Establish the basic development environment and repository structure.

Tasks:

- Initialize frontend and backend projects.
- Configure TypeScript, Python, formatting, linting, and basic testing.
- Set up PostgreSQL/Supabase.
- Establish authentication.
- Create environment-variable and secret management.
- Create initial database migrations.
- Establish API versioning.
- Configure development and test environments.
- Set up basic logging and error handling.

- Establish the LLM provider abstraction.
- Create the initial deployment pipeline.

Exit condition: A user can sign in, reach the application, and communicate with the backend through the basic API.

4 Milestone 1 — Conversational Core

Build the first usable HeeraMon interaction.

Tasks:

- Implement the main conversational interface.
- Add HeeraMon's initial personality and onboarding behavior.
- Accept natural-language job/life requirements.
- Extract structured preferences from conversation.
- Classify preferences as hard, soft, learned, or unknown.
- Ask clarification questions when necessary.
- Persist confirmed preferences.
- Display the understood preferences to the user.

Exit condition: A new user can describe what they want and see an understandable representation of what HeeraMon believes they want.

5 Milestone 2 — Job Ingestion

Build the first real job discovery pipeline.

Tasks:

- Implement the job-source adapter interface.
- Integrate a small number of accessible and reliable sources.
- Fetch job postings.
- Normalize job data.
- Validate records.
- Deduplicate postings.
- Store source information and freshness data.
- Handle source failures independently.
- Add ingestion tests.

Do not optimize for the number of sources yet.

Exit condition: HeeraMon can repeatedly discover real jobs and store them in a con-

sistent internal format without uncontrolled duplication.

6 Milestone 3 — Matching and Job Presentation

Connect user preferences with real job data.

Tasks:

- Implement hard-constraint evaluation.
- Implement soft-preference comparison.
- Handle unknown information explicitly.
- Identify meaningful trade-offs.
- Add LLM-assisted job interpretation where necessary.
- Generate understandable match explanations.
- Build the job opportunity interface.
- Allow users to inspect job details.
- Record user decisions.

Exit condition: A user can receive a real job and understand why HeeraMon considers it relevant, including what does not match.

7 Milestone 4 — Continuous Search

Turn HeeraMon from a search interface into a persistent assistant.

Tasks:

- Implement background scheduling.
- Run ingestion independently of the browser session.
- Evaluate newly discovered jobs.
- Detect meaningful matches.
- Generate notifications.
- Add in-app notifications.
- Add email notifications if practical.
- Implement notification deduplication.
- Add pause/resume and search-frequency controls.
- Add retry and idempotency handling.
- Record background task activity.

Exit condition: HeeraMon can continue searching after the user leaves the application and notify the user when a meaningful opportunity appears.

8 Milestone 5 — Preference Learning

Make HeeraMon progressively more useful without silently taking control.

Tasks:

- Record user decisions.
- Detect repeated behavioral patterns.
- Distinguish observations from confirmed preferences.
- Ask the user before promoting an important inferred preference.
- Persist confirmed learned preferences.
- Incorporate learned preferences into future matching.
- Explain when previous decisions influence a recommendation.

Exit condition: User decisions can improve future recommendations while the user remains aware of important preference changes.

9 Milestone 6 — Application Assistance

Add the application workflow after discovery and matching are stable.

Tasks:

- Store user application/profile information.
- Support CV management.
- Prepare application information from known data.
- Generate drafts such as cover letters or application answers where appropriate.
- Identify missing information.
- Prevent fabricated qualifications or facts.
- Build the application review interface.
- Implement Edit / Cancel / Approve actions.
- Add explicit backend approval enforcement.
- Integrate a small number of supported application paths.
- Record application status and history.

Exit condition: A user can select a supported job, review a prepared application, explicitly approve it, and receive an accurate submission status.

10 Milestone 7 — Hardening and Release

Stabilize the complete product before expanding scope.

Tasks:

- Run unit, integration, agent, security, and end-to-end tests.
- Test major failure and recovery paths.
- Review authorization and user-data isolation.
- Test prompt-injection and tool-boundary protections.
- Improve logging and observability.
- Review notification behavior.
- Remove unnecessary complexity and unused dependencies.
- Improve loading, empty, and error states.
- Perform basic accessibility checks.
- Verify deployment and recovery procedures.
- Run a small real-user/beta test.

Exit condition: The complete core loop works reliably for a small number of users and important failures are understandable and recoverable.

11 Recommended Build Order

The implementation order is:

1. **Foundation** — Authentication + Database + API
2. **Conversational Core** — Preferences + HeeraMon interaction
3. **Job Discovery** — Sources + Normalization + Deduplication
4. **Matching** — Constraints + Trade-offs + Explanations
5. **Continuous Assistance** — Background Search + Notifications
6. **Learning** — Decisions + Confirmed Preference Learning
7. **Application** — Preparation + Review + Approval + Supported Submission
8. **Hardening** — Testing + Security + Deployment + Beta

This order intentionally delays application automation until the discovery and matching loop is already useful.

12 Definition of Completion

The initial release is complete when a real user can:

1. Sign in.
2. Tell HeeraMon what kind of work and life they want.
3. Confirm or correct HeeraMon's understanding.
4. Have those preferences persist.
5. Receive real job opportunities.

6. Understand why an opportunity fits or does not fit.
7. Investigate a job conversationally.
8. Accept or reject opportunities.
9. Have confirmed decisions influence future recommendations.
10. Receive useful notifications from background searching.
11. Select a supported opportunity for application.
12. Review and edit the prepared application.
13. Explicitly approve or cancel submission.
14. Receive an accurate application result.

13 What Should Not Delay Release

The following should not become prerequisites for the initial release:

- Large-scale job-source coverage.
- Multi-agent architecture.
- Sophisticated ML recommendation models.
- Vector databases.
- Kubernetes or microservices.
- Native mobile applications.
- Universal browser automation.
- Complex analytics dashboards.
- Subscription and billing systems.
- Advanced personality modeling.
- Perfect commute estimation.
- Fully autonomous application submission.

If the core loop works without one of these, it belongs after the initial release.

14 Development Rule

At every stage, ask:

“Does this make the complete job-seeking loop more real, reliable, or useful?”

If not, defer it unless it is required for security, correctness, or maintainability.

Guiding Roadmap Principle: Build the smallest complete HeeraMon, not the largest incomplete one.

15 Document Control

Version	Date	Author / Role	Changes
1.0	September 30, 2026	Product / Engineering	Initial release

Approval: This document defines the development sequence for HeeraMon. All subsequent planning and implementation work must remain consistent with the principles stated herein.

HeeraMon flies to help people find job that fits not only their career, but the life they want to live.



Saadman Sakib
Product Engineer